

GPT-RAG Sensengo KT

GPT-RAG Sensengo 專案 Knowledge Transfer 文件

專案名稱: GPT-RAG Sensengo (林口恩典大樓企業資訊 Agent)

客戶: 東森集團企業 (sensengo.com.tw)

建立日期: 2026年2月6日

版本: 1.0

目錄

- 專案概覽
- 系統架構
- 部署指南
- 開發指南
- 疑難排解
- 附錄

1. 專案概覽

1.1 專案背景

東森集團正在興建「林口恩典大樓」(Grace Tower)，包含辦公室、酒店、餐廳、商場、教堂、展演空間等設施。此專案需要一個企業級 RAG (Retrieval-Augmented Generation) 系統，協助內部員工查詢專案相關資訊並進行商業規劃。

1.2 專案目標

目標	說明
正確性與創造性平衡	Agent 需能精確回答資料，同時具備創意發想能力
多模態支援	處理 Word、PowerPoint、Excel、圖片等多種檔案格式
檔案管理	支援上傳、刪除、版本控管
跨裝置存取	支援桌面、手機、平板等多種裝置

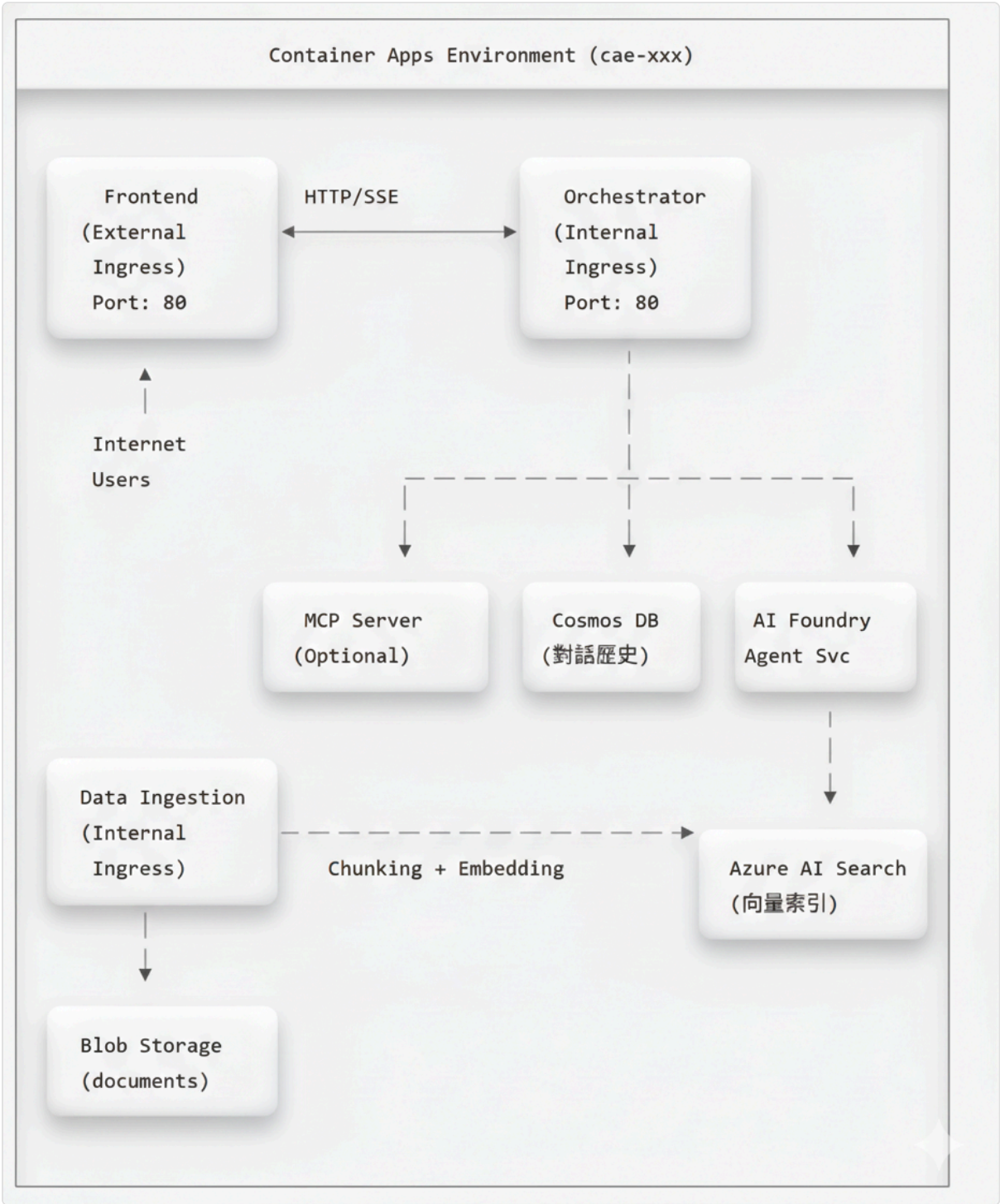
1.3 技術選型

本專案採用 Microsoft GPT-RAG 解決方案加速器，基於以下技術：

層級	技術
AI 框架	Azure AI Foundry Agent Service (azure-ai-agents>=1.2.0b4)
LLM 模型	GPT-5.2 (GlobalStandard)
向量搜尋	Azure AI Search (azure-search-documents 11.5~11.7)
Embedding	text-embedding-3-large
後端框架	FastAPI 0.115+ / Uvicorn 0.34+ / Python 3.12
前端框架	Chainlit 2.6.0
AI 擴展	Semantic Kernel 1.34+ (含 MCP 支援)
容器運行	Azure Container Apps (Consumption)
IaC 工具	Bicep + Azure Developer CLI (azd 1.22+)

2. 系統架構

2.1 高層架構圖



高層架構圖

2.2 Azure 資源清單

資源類型	命名規則	SKU/層級	用途
AI Foundry	aif- {token} -GPRAG	Standard	AI 模型管理平台
Azure OpenAI	(AI Foundry 內建)	GlobalStandard	LLM 推理
Azure AI Search	srch- {token} -GPRAG	Basic	向量與混合搜尋
Cosmos DB	cosmos- {token} -GPRAG	Serverless	對話歷史儲存
Container Apps	ca- {token} -*-GPRAG	Consumption	微服務運行
Container Apps Env	cae- {token} -GPRAG	-	共用環境
Container Registry	cr {token} GPRAG	Standard	Docker 映像
Storage Account	st {token} GPRAG	Standard LRS	文檔儲存
App Configuration	appcs- {token} -GPRAG	Standard	集中配置
Key Vault	kv- {token} -GPRAG	Standard	密鑰管理
Log Analytics	log- {token} -GPRAG	Pay-as-you-go	日誌收集
Application Insights	appi- {token} -GPRAG	Pay-as-you-go	應用監控

Token 說明: {token} 是由 azd 自動產生的唯一識別碼，例如 2v3lftkn4xam

2.3 Container Apps 服務

服務	Container App 名稱	功能	Ingress
Frontend	ca- {token} -frontend-GPRAG	Chainlit 聊天介面	External
Orchestrator	ca- {token} -orch-GPRAG	RAG 核心引擎	Internal
DataIngest	ca-ingest-GPRAG	文件處理與索引	Internal
MCP	ca- {token} -mcp-GPRAG	工具擴充 (選用)	Internal

2.4 AI 模型部署

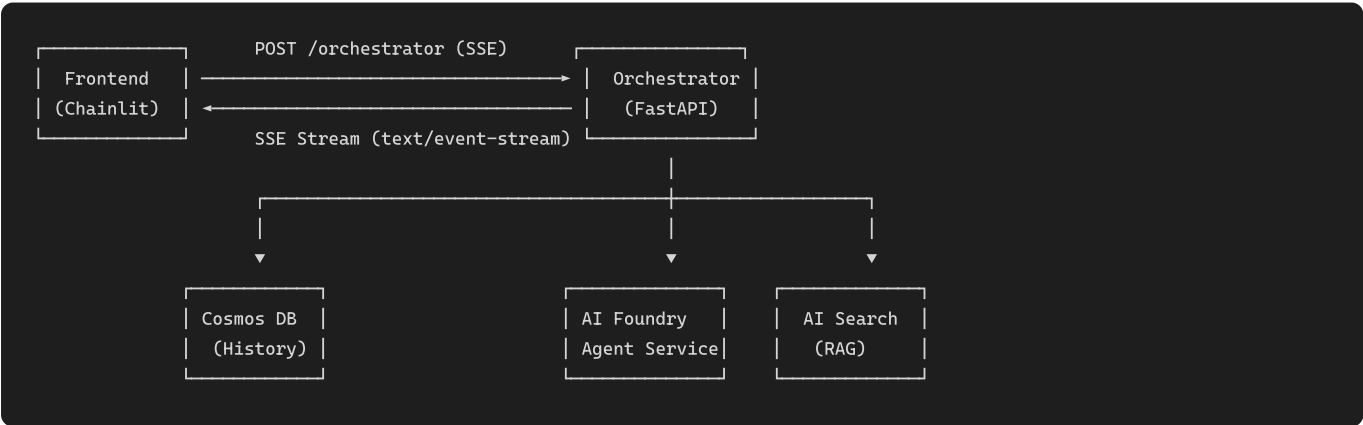
模型	部署名稱	SKU	容量 (TPM)	用途
GPT-5.2	chat	GlobalStandard	80K	對話生成
text-embedding-3-large	text-embedding	Standard	40K	向量嵌入

2.5 資料流程

查詢處理流程



API Flow:



查詢處理 Function 呼叫鏈:

以下描述每個 Function 被誰呼叫、輸入什麼、內部再呼叫誰、輸出什麼：

① `app.handle_message()` — 使用者訊息進入點

項目	說明
檔案	<code>gpt-rag-ui/app.py</code> (Chainlit <code>@cl.on_message</code> handler)
呼叫者	Chainlit 框架 (使用者在聊天介面送出訊息時自動觸發)
輸入	<code>cl.Message</code> 物件，包含 <code>message.content</code> (使用者問題)、 <code>message.id</code>
內部處理	1. 取得 <code>conversation_id</code> 、 <code>auth_info</code> 、 <code>debug_mode</code> 、 <code>search_index</code> 等 session 參數 2. 呼叫 ② <code>call_orchestrator_stream()</code>
輸出	SSE 串流回應，逐 chunk 透過 <code>response_msg.stream_token()</code> 顯示在 Chainlit UI

② `call_orchestrator_stream()` — Frontend → Orchestrator HTTP Client

項目	說明
檔案	<code>gpt-rag-ui/orchestrator_client.py</code>
呼叫者	<code>app.handle_message()</code>
輸入	<code>conversation_id</code> ， <code>question</code> (str), <code>auth_info</code> (dict), <code>question_id</code> ， <code>debug_mode</code> ， <code>search_index</code>
內部處理	1. 讀取 <code>ORCHESTRATOR_BASE_URL</code> 或組合 Dapr sidecar URL 2. 組裝 HTTP headers (<code>X-API-KEY</code> / <code>dapr-api-token</code>) 3. 組裝 JSON payload: <code>{ask, question, conversation_id, client_principal_id, client_principal_name, debug_mode, search_index, ... }</code> 4. 使用 <code>httpx.AsyncClient.stream("POST", url, json=payload)</code> 發送請求 5. 呼叫 ③ <code>POST /orchestrator</code>
輸出	<code>AsyncIterator[str]</code> — SSE 文字串流 (每個 chunk yield 回給呼叫者)

③ `orchestrator_endpoint()` — FastAPI Endpoint

項目	說明
檔案	<code>gpt-rag-orchestrator/src/main.py</code>
呼叫者	<code>call_orchestrator_stream()</code> 透過 HTTP POST
輸入	<code>OrchestratorRequest</code> (Pydantic model · 定義在 <code>schemas.py</code>) : - <code>ask</code> (str, 必填) — 使用者問題 - <code>conversation_id</code> (str, 選填) — 對話 ID - <code>debug_mode</code> (bool, 選填) — 啟用 debug - <code>search_index</code> (str, 選填) — 指定 AI Search 索引 - <code>type</code> (str, 選填) — <code>"feedback"</code> 則走回饋路徑 - <code>question_id</code>, <code>client_principal_id</code>, <code>client_principal_name</code>, <code>client_group_names</code>, <code>access_token</code>, <code>user_context</code> 等
內部處理	1. 驗證認證 (<code>validate_auth</code> dependency, 檢查 <code>X-API-KEY</code> 或 <code>dapr-api-token</code>) 2. 若 <code>type == "feedback"</code> → 呼叫 <code>orchestrator.save_feedback()</code> 後直接回傳 3. 否則呼叫 ④ <code>Orchestrator.create()</code> 建立實例 4. 呼叫 ⑤ <code>orchestrator.stream_response(ask)</code> 取得回應串流 5. 包裝為 <code>StreamingResponse(media_type="text/event-stream")</code>
輸出	<code>StreamingResponse</code> — SSE 串流 · 包含 <code>conversation_id</code> + 回應文字 + debug events

④ `Orchestrator.create()` — 建立 Orchestrator 實例

項目	說明
檔案	<code>gpt-rag-orchestrator/src/orchestration/orchestrator.py</code>
呼叫者	<code>orchestrator_endpoint()</code>
輸入	<code>conversation_id</code> , <code>user_context</code> (dict), <code>debug_mode</code> (bool), <code>search_index</code> (str)
內部處理	1. 初始化 <code>CosmosDBClient</code> (對話歷史存取) 2. 從 App Configuration 讀取 <code>AGENT_STRATEGY</code> (預設 <code>"single_agent_rag"</code>) 3. 呼叫 <code>AgentStrategyFactory.get_strategy(name)</code> → 得到 Strategy 實例 4. 將 <code>debug_mode</code> 、 <code>search_index</code> 設定到 Strategy 上
輸出	<code>Orchestrator</code> 實例 (含已初始化的 <code>agentic_strategy</code>)

`AgentStrategyFactory.get_strategy()` 對照表 (定義在 `strategies/agent_strategy_factory.py`) :

Key	對應 Class	檔案
<code>single_agent_rag</code>	<code>SingleAgentRAGStrategyV1</code>	<code>strategies/single_agent_rag_strategy_v1.py</code>
<code>mcp</code>	<code>McpStrategy</code>	<code>strategies/mcp_strategy.py</code>
<code>nl2sql</code>	<code>NL2SQLStrategy</code>	<code>strategies/nl2sql_strategy.py</code>

⑤ `Orchestrator.stream_response(ask)` — 核心協調流程

項目	說明
檔案	<code>gpt-rag-</code> <code>orchestrator/src/orchestration/orchestrator.py</code>
呼叫者	<code>orchestrator_endpoint()</code> (在 SSE generator 中)
輸入	<code>ask</code> (str 使用者問題), <code>question_id</code> (str, 選填)
內部處理	1. 從 Cosmos DB 載入或建立 conversation document 2. 記錄 question_id 到 conversation 3. 將 conversation 傳給 strategy 4. 呼叫 ⑥ <code>strategy.initiate_agent_flow(ask)</code> 取得回應串流 5. 完成後更新 conversation document 至 Cosmos DB
輸出	<code>AsyncIterator[str]</code> — 先 yield <code>{conversation_id}</code> · 再 yield 所有 strategy 回應 chunk

⑥ `SingleAgentRAGStrategyV1.initiate_agent_flow(ask)` — Agent 執行流程

項目	說明
檔案	<code>gpt-rag-</code> <code>orchestrator/src/strategies/single_agent_rag_strategy_v1.py</code>
呼叫者	<code>Orchestrator.stream_response()</code>
輸入	<code>user_message</code> (str 使用者問題)
內部處理 (依序)	Step 1: <code>_get_or_create_thread()</code> — 建立或取回 AI Foundry Agent Thread Step 2: <code>_get_or_create_agent()</code> — 建立 Agent (含 system prompt, tools 定義) Step 3: <code>_send_user_message()</code> — 將 user_message 送入 Thread Step 4: <code>_stream_agent_response()</code> → 呼叫 ⑦ Agent Run Stream Step 5: <code>_consolidate_conversation_history()</code> — 從 Thread 取回完整對話歷史 Step 6: <code>_cleanup_agent()</code> — 刪除暫時 Agent
內部 Tools	Agent 初始化時註冊的 FunctionTools : - ⑧ <code>SearchClient.search_knowledge_base(query)</code> — RAG 知識庫搜尋 - ⑨ <code>CallTranscriptClient.query_call_transcripts(...)</code> — 通話記錄查詢 (若啟用) - <code>BingGroundingTool</code> — Bing 搜尋 (若啟用)
輸出	<code>AsyncIterator[str]</code> — Agent 回應文字 chunk + debug events (若 debug mode)

⑦ `_stream_agent_response()` — Agent Run 串流

項目	說明
檔案	<code>gpt-rag-orchestrator/src/strategies/single_agent_rag_strategy_v1.py</code>
呼叫者	<code>initiate_agent_flow()</code> Step 4
輸入	<code>project_client</code> , <code>agent_id</code> , <code>thread_id</code> , <code>user_message</code>
內部處理	1. 呼叫 <code>project_client.agents.runs.stream(thread_id, agent_id)</code> 啟動 Agent Run 2. LLM 第一次思考 → 決定需要呼叫哪些 Tools 3. Auto-execute registered Tools (⑥ 或 ⑨) — SDK 自動執行 4. LLM 第二次思考 → 基於 Tool 結果生成最終回應 5. 處理 <code>thread.message.delta</code> 事件 · 逐 chunk yield 回應文字
輸出	<code>AsyncIterator[str]</code> — 回應文字 chunk (含引用處理)

⑧ `SearchClient.search_knowledge_base(query)` — RAG 知識庫搜尋

項目	說明
檔案	<code>gpt-rag-orchestrator/src/connectors/search.py</code>
呼叫者	AI Foundry Agent SDK auto-execute (當 Agent 決定需要搜尋知識庫時)
輸入	<code>query</code> (str) — Agent 產生的搜尋查詢
內部處理	1. 依 <code>search_approach</code> (hybrid/vector/term) 組裝搜尋 body 2. 若 vector/hybrid → 呼叫 Azure OpenAI <code>get_embeddings(query)</code> 產生向量 3. 呼叫 Azure AI Search REST API 執行搜尋 4. 解析結果取 <code>title</code> , <code>content</code> , <code>url</code> , <code>filepath</code>
輸出	JSON string — <code>[{title, link, content}, ...]</code> 搜尋結果列表

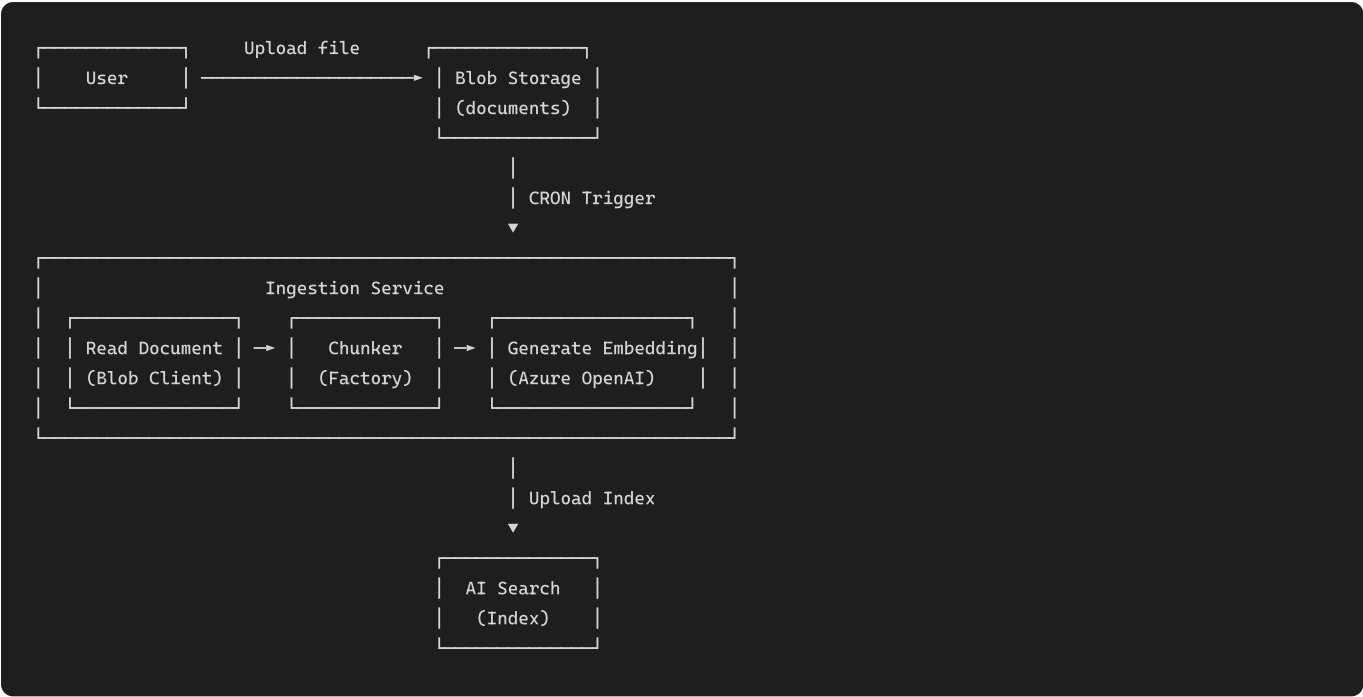
⑨ `CallTranscriptClient.query_call_transcripts(...)` — 通話記錄查詢

項目	說明
檔案	<code>gpt-rag-orchestrator/src/connectors/call_transcripts.py</code>
呼叫者	AI Foundry Agent SDK auto-execute (當 Agent 決定需要查詢通話記錄時)
輸入	<code>customer_id</code> (str, 選填), <code>status</code> (str, 選填: “成功” / “失敗”), <code>call_date</code> (str, 選填: “YYYY-MM-DD”), <code>keyword</code> (str, 選填), <code>top</code> (int, 預設 10), <code>include_full_transcript</code> (str, 預設 “false”)
內部處理	1. 動態組裝 Cosmos DB SQL 查詢 (WHERE 條件) 2. 透過 <code>CosmosClient</code> 查詢 <code>call-transcripts</code> container 3. 格式化結果 (截斷 transcript 至前 300 字元 · 除非 <code>include_full_transcript=true</code>)
輸出	JSON string — 包含符合條件的通話記錄列表

文件 Ingestion 流程

```
1. 使用者上傳文件到 Blob Storage (documents container)
  ↓
2. Ingestion Service 依 CRON 排程觸發
  ↓
3. 讀取文件並依類型選擇 Chunker
   ├── PDF/DOCX/PPTX → Document Intelligence
   ├── XLSX → SpreadsheetChunker
   └── 其他 → LangChain Chunker
  ↓
4. 文件切分成 Chunks (預設 2048 tokens)
  ↓
5. Azure OpenAI 生成向量嵌入 (text-embedding-3-large)
  ↓
6. Chunks + Vectors 上傳到 Azure AI Search
  ↓
7. 寫入 Job 執行記錄
```

API Flow:



Ingestion Function 呼叫鏈 (CRON 排程路徑):

以下描述 Blob 索引排程 (最主要路徑) 中每個 Function 的呼叫關係：

1 lifespan() → Scheduler 啟動 — 應用程式啟動入口

項目	說明
檔案	<code>gpt-rag-ingestion/main.py</code>
呼叫者	FastAPI 框架 (應用程式啟動時自動執行)
輸入	無 (讀取 App Configuration 中的 CRON 設定)
內部處理	1. 驗證 Azure 認證 (Managed Identity / Service Principal / az login) 2. 初始化 App Configuration Client 3. 讀取各 <code>CRON_RUN_*</code> 設定，透過 APScheduler <code>CronTrigger</code> 註冊排程 4. 啟動時立即執行一次已排程的 Jobs (如 2 <code>run_blob_index()</code>)
輸出	無 (排程在背景持續運行)

排程 Job 對應表：

CRON Key	呼叫 Function	對應 Class
CRON_RUN_BLOB_INDEX	run_blob_index()	BlobStorageDocumentIndexer
CRON_RUN_BLOB_PURGE	run_blob_purge()	BlobStorageDeletedItemsCleaner
CRON_RUN_SHAREPOINT_INDEX	run_sharepoint_index()	SharePointIndexer
CRON_RUN_NL2SQL_INDEX	run_nl2sql_index()	NL2SQLIndexer

2 run_blob_index() — Blob 索引排程觸發點

項目	說明
檔案	gpt-rag-ingestion/main.py
呼叫者	APScheduler CRON 觸發 或 lifespan 啟動時立即呼叫
輸入	無
內部處理	實例化 BlobStorageDocumentIndexer() 並呼叫 3 .run()
輸出	無 (執行結果記錄在 Blob Storage logs)

3 BlobStorageDocumentIndexer.run() — Blob 索引核心邏輯

項目	說明
檔案	<code>gpt-rag-ingestion/jobs/blob_storage_indexer.py</code>
呼叫者	<code>run_blob_index()</code>
輸入	<code>BlobIndexerConfig</code> (從 App Configuration 讀取) : <code>storage_account_name</code> , <code>source_container</code> , <code>search_endpoint</code> , <code>search_index_name</code> , <code>max_concurrency</code> 等
內部處理	1. <code>_ensure_clients()</code> — 建立 <code>BlobServiceClient</code> + <code>AsyncSearchClient</code> 2. <code>_load_latest_index_state()</code> — 從 AI Search 載入現有文件的 <code>last_modified</code> map 3. 列舉 Blob Storage container 中所有文件 4. 比對 <code>blob.last_modified</code> > <code>prev_last_modified</code> → 篩出需重新索引的文件 5. 並行呼叫 ④ <code>_process_one()</code> 處理每個文件 (受 <code>max_concurrency</code> 限制) 6. 寫入 run summary 至 Blob Storage jobs log container
輸出	Summary dict: <code>{sourceFiles, candidates, indexedItems, success, failed, totalChunksUploaded}</code>

④ `_process_one(blob_name, last_modified, content_type)` — 單一文件處理

項目	說明
檔案	<code>gpt-rag-ingestion/jobs/blob_storage_indexer.py</code>
呼叫者	<code>BlobStorageDocumentIndexer.run()</code> (並行呼叫)
輸入	<code>blob_name</code> (str), <code>last_modified</code> (datetime), <code>content_type</code> (str), <code>run_id</code> (str)
內部處理	1. 從 Blob Storage 下載文件 bytes (<code>blob_client.download_blob()</code>) 2. 讀取 blob metadata 中的 <code>security_ids</code> (文件權限控制) 3. 組裝 <code>data = {documentUrl, documentContentType, documentBytes, fileName}</code> 4. 呼叫 ❸ <code>DocumentChunker().chunk_documents(data)</code> 進行文件切分 5. 將 chunks 轉換為 AI Search documents (<code>_to_search_doc()</code>) 6. 呼叫 <code>_replace_parent_docs()</code> — 刪除舊 chunks 後上傳新 chunks 至 AI Search 7. 寫入 per-file log
輸出	<code>{status: "success", chunks: N}</code> 或 <code>{status: "error", error: " ... "}</code>

❸ `DocumentChunker.chunk_documents(data)` — 文件切分入口

項目	說明
檔案	<code>gpt-rag-ingestion/chunking/document_chunking.py</code>
呼叫者	<code>_process_one()</code> 或 <code>/document-chunking</code> HTTP endpoint
輸入	<code>data</code> (dict): <code>{documentUrl, documentContentType, documentBytes, fileName}</code>
內部處理	1. 呼叫 ❹ <code>ChunkerFactory().get_chunker(data)</code> 取得對應的 Chunker 2. 呼叫 <code>chunker.get_chunks()</code> 執行實際切分
輸出	<code>(chunks, errors, warnings)</code> — chunks 為 dict list · 每個含 <code>content</code> , <code>title</code> , <code>url</code> 等

❹ `ChunkerFactory.get_chunker(data)` — Chunker 工廠

項目	說明
檔案	<code>gpt-rag-ingestion/chunking/chunker_factory.py</code>
呼叫者	<code>DocumentChunker.chunk_documents()</code>
輸入	<code>data</code> (dict) — 包含 <code>fileName</code> 用以判斷副檔名
內部處理	依檔案副檔名選擇 Chunker 實例
輸出	對應的 Chunker 實例

Chunker 對照表：

副檔名	Chunker Class	說明
<code>pdf</code> , <code>png</code> , <code>jpeg</code> , <code>jpg</code> , <code>bmp</code> , <code>tiff</code>	<code>DocAnalysisChunker</code>	透過 Document Intelligence 分析
<code>docx</code> , <code>pptx</code>	<code>DocAnalysisChunker</code>	需 Doc Intelligence 4.0 API
<code>xlsx</code> , <code>xls</code>	<code>SpreadsheetChunker</code>	試算表切分
<code>vtt</code>	<code>TranscriptionChunker</code>	字幕/逐字稿
<code>json</code>	<code>JSONChunker</code>	JSON 資料切分
<code>nl2sql</code>	<code>NL2SQLChunker</code>	NL2SQL schema 切分
其他 (<code>txt</code> , <code>md</code> 等)	<code>LangChainChunker</code>	LangChain 通用文字切分

若 `MULTIMODAL=true` , PDF/圖片/DOCX/PPTX 會改用 `MultimodalChunker` 。

Ingestion HTTP Endpoints 呼叫鏈 (手動觸發路徑):

除了 CRON 排程 , 也可透過 HTTP API 手動觸發：

7 `POST /document-chunking` — 手動文件切分

項目	說明
檔案	<code>gpt-rag-ingestion/main.py</code>
呼叫者	Azure AI Search Skillset 或手動 HTTP 呼叫
認證	API Key (<code>validate_api_key_header</code>)
輸入	JSON Body: <code>{"values": [{"recordId": "1", "data": {"documentUrl": "https:// ...", "documentContentType": "application/pdf"}}]}</code>
內部處理	1. JSON Schema 驗證 2. 透過 <code>BlobClient</code> 下載文件 bytes 3. 呼叫 ⑤ <code>DocumentChunker().chunk_documents(data)</code> 4. 組裝回應
輸出	JSON: <code>{"values": [{"recordId": "1", "data": {"chunks": [...]}, "errors": [], "warnings": []}]}</code>

⑧ `POST /text-embedding` — 手動向量嵌入

項目	說明
檔案	<code>gpt-rag-ingestion/main.py</code>
呼叫者	Azure AI Search Skillset 或手動 HTTP 呼叫
認證	API Key (<code>validate_api_key_header</code>)
輸入	JSON Body: <code>{"values": [{"recordId": "1", "data": {"text": "要嵌入的文字"}}]}</code>
內部處理	1. 實例化 <code>AzureOpenAIClient()</code> 2. 對每個 item 呼叫 <code>aoai_client.get_embeddings(text)</code> (Azure OpenAI text-embedding-3-large) 3. 組裝回應
輸出	JSON: <code>{"values": [{"recordId": "1", "data": {"embedding": [0.012, -0.034, ...]}, "errors": [], "warnings": []}]}</code>

3. 部署指南

3.1 先決條件

工具安裝

工具	版本	安裝指令
Azure CLI	2.80+	<code>winget install Microsoft.AzureCLI</code>
Azure Developer CLI	1.22+	<code>winget install Microsoft.Azd</code>
Docker	29+	Docker Desktop
Python	3.12	<code>winget install Python.Python.3.12</code>
Git	最新	<code>winget install Git.Git</code>

Azure 權限

- 訂閱層級 **Owner** 或 **Contributor + User Access Administrator**
- 需註冊以下 Resource Provider :
 - `Microsoft.AppConfiguration`
 - `Microsoft.DocumentDB`
 - `Microsoft.ContainerService`
 - `Microsoft.CognitiveServices`

3.2 部署步驟

Step 1: 登入 Azure

```
# 登入指定租戶
az login --tenant {tenant-id}

# 設定訂閱
az account set --subscription "{subscription-name}"

# 驗證權限
az role assignment list --assignee $(az ad signed-in-user show --query id -o tsv) --query "[].roleDefinitionName" -o table
```

Step 2: 設定 azd 環境

```
cd GPT-RAG

# 初始化環境
azd init -e {environment-name}

# 設定區域
azd env set AZURE_LOCATION eastus2
```

Step 3: 佈建基礎設施

```
# 執行 Bicep 部署
azd provision
```

預期結果：約 20-25 個 Azure 資源建立完成

Step 4: 部署應用程式

```
# 部署所有服務
azd deploy
```

如果 `azd deploy` 失敗，可手動部署：

```
# 登入 ACR
az acr login --name cr{token}

# 建置並推送 Frontend
cd ../gpt-rag-ui
$ts = Get-Date -Format "yyyyMMddHHmmss"
docker build -t cr{token}.azurecr.io/azure-gpt-rag/frontend:$ts .
docker push cr{token}.azurecr.io/azure-gpt-rag/frontend:$ts

# 更新 Container App
az containerapp update --name ca-{token}-frontend --resource-group GPRAG `
  --image cr{token}.azurecr.io/azure-gpt-rag/frontend:$ts
```

3.3 部署後驗證

驗證清單

- ☐ Container Apps 狀態為 Running
- ☐ Frontend URL 可正常存取
- ☐ AI Search 索引已建立
- ☐ App Configuration 參數正確

驗證指令

```
# 檢查 Container Apps 狀態
az containerapp list --resource-group GPRAG --query "[].{name:name, state:properties.runningStatus}" -o table

# 檢查 AI Search 索引
$searchEndpoint = "https://srch-{token}.search.windows.net"
$token = az account get-access-token --resource "https://search.azure.com" --query accessToken -o tsv
Invoke-RestMethod -Uri "$searchEndpoint/indexes?api-version=2023-11-01" `
  -Headers @{ "Authorization" = "Bearer $token" }
```

4. 開發指南

4.1 Debug 面板功能

Frontend 內建 Debug 面板，可顯示詳細的執行資訊：

啟用方式： - 預設為啟用 - 輸入 `/debug off` 可關閉 - 輸入 `/debug` 或 `/debug on` 重新啟用

顯示內容：

面板	內容
Timing	各階段執行時間 (Thread/Agent 管理、LLM 思考、工具執行等)
Prompting Details	System Prompt、Tool Calls、Search Results、LLM Calls

UI 佈局：左右分割 (問答區 55%、Debug Panel 45%)

4.2 Agent 策略切換

系統支援三種 Agent 策略：

策略	設定值	說明
Single Agent RAG	<code>single_agent_rag</code>	預設模式，單一 Agent + RAG 工具
MCP	<code>mcp</code>	Model Context Protocol 擴充工具
NL2SQL	<code>nl2sql</code>	自然語言轉 SQL 查詢

切換方式：在 App Configuration 設定 `AGENT_STRATEGY`

```
az appconfig kv set --endpoint "https://appcs-{token}.azconfig.io" `
  --key "AGENT_STRATEGY" --value "single_agent_rag" --label "gpt-rag" --auth-mode login -y
```

4.3 設定參數清單

核心設定

Key	說明	預設值
AGENT_STRATEGY	Agent 策略	single_agent_rag
CHAT_DEPLOYMENT_NAME	LLM 部署名稱	chat
EMBEDDING_DEPLOYMENT_NAME	Embedding 部署名稱	embedding
SEARCH_RAGINDEX_TOP_K	搜尋結果數量	3
SEARCH_APPROACH	搜尋方法 (hybrid/vector/term)	hybrid
ORCHESTRATOR_URI	Orchestrator 內部 URL	(自動設定)

Ingestion 設定

Key	說明	預設值
CRON_RUN_BLOB_INDEX	Blob 索引排程	0 */6 * * *
CRON_RUN_BLOB_PURGE	Blob 清理排程	0 0 * * *
RUN_JOBS_ON_STARTUP	啟動時執行 Job	true
DOCUMENTS_STORAGE_CONTAINER	文件容器名稱	documents
SEARCH_RAG_INDEX_NAME	搜尋索引名稱	ragindex-{token}

Chunking 設定

Key	說明	預設值
CHUNKING_NUM_TOKENS	一般文件 Chunk 大小	2048
TOKEN_OVERLAP	Chunk 重疊 tokens	100
CHUNKING_MIN_CHUNK_SIZE	最小 Chunk 大小	100
SPREADSHEET_CHUNKING_NUM_TOKENS	Excel Chunk 大小	0 (無限制) ⚠
SPREADSHEET_CHUNKING_BY_ROW	按列切分 Excel	false

⚠ 注意: SPREADSHEET_CHUNKING_NUM_TOKENS=0 表示不限制大小，可能產生超大 Chunk (如 31,772 字元)。建議設為 2048。

Chunker 對應表

檔案類型	Chunker
<code>.pdf</code> , <code>.docx</code> , <code>.pptx</code> , <code>.png</code> , <code>.jpg</code>	DocAnalysisChunker (使用 Document Intelligence)
<code>.xlsx</code> , <code>.xls</code>	SpreadsheetChunker
<code>.json</code>	JSONChunker
<code>.vtt</code>	TranscriptionChunker
<code>.md</code> , <code>.txt</code> , <code>.html</code> , <code>.py</code>	LangChainChunker

5. 疑難排解

5.1 常見問題

問題 1: Frontend 顯示 “An internal server error occurred.”

可能原因: 1. Cosmos DB 防火牆阻擋 2. TPM 配額用完 3. Orchestrator 連線失敗

排查步驟:

```
# 1. 檢查 Cosmos DB 網路設定
az cosmosdb show --name cosmos-{token} --resource-group GPRAG `
  --query "publicNetworkAccess"

# 2. 檢查 Orchestrator 日誌
az containerapp logs show --name ca-{token}-orch --resource-group GPRAG --tail 100
```

問題 2: Ingestion 失敗 - AuthorizationFailure

原因: Storage Account `publicNetworkAccess` 設為 Disabled

解決:

```
az storage account update --name st{token} --resource-group GPRAG `
  --public-network-access Enabled
```

問題 3: Container App OOM Killed

症狀: Container 頻繁重啟 · Exit Code 137

解決: 增加記憶體配置

```
az containerapp update --name ca-ingest-gprag --resource-group GPRAG `
  --cpu 1.0 --memory 2Gi
```

問題 4: 回應延遲過長 (~43 秒)

分析: 這是 Agent 架構的正常現象

階段	時間	說明
Cosmos DB	~4s	載入對話歷史
Agent 思考	~7s	決定呼叫工具
RAG 檢索	~1.5s	搜尋 + Embedding
LLM 生成	~27s	主要瓶頸

優化建議: - 使用較小的模型 (如 GPT-4.1 Mini) - 減少 `SEARCH_RAGINDEX_TOP_K` - 調整 Chunk 大小 — 降低 `CHUNK_SIZE` (如從 2048 降至 1024 tokens) · 使每個 chunk 內容更精簡 · 減少 LLM 輸入 token 數量 · 從而加速回應生成 - 簡化 System Prompt

6. 附錄

6.1 重要聯絡資訊

角色	說明
Azure 訂閱	ehs-ai-lab (2c9b3248-f263-4104-bd24-6446d4db84b9)
Tenant	東森集團企業 (45f5172d-7608-4bd1-a52a-c3a7de0423d3)
Resource Group	GPRAG
部署區域	East US 2

6.2 版本歷史

版本	日期	變更說明
1.0	2026-02-06	初版 KT 文件